



Python Fundamentals III

Data Analytics Bootcamp



Today

1. **Objects & Collections**
 - a. What Are Methods?
2. **Lists**
 - a. List Syntax
 - b. List Operations
3. **Dictionaries**
 - a. Dictionary Syntax
 - b. Dictionary Operations
4. **Loops**
 - a. For Loops
 - b. While Loops
 - c. **Exercise:** Python Loops



Topic

Objects & Collections



Objects

An **object** is a fundamental concept representing a self-contained unit that consists of both data and functionality (**methods**).

- **Methods** are a built-in function specific to a type of object. They are typically the most common actions you can expect to take.



Collections

A **collection** is an object that holds multiple items in a single variable. Python has several built-in collection types, today you will meet:

- **Lists**
- **Dictionaries**



Topic Lists



Lists

Lists allow you to store multiple items in a **single variable**.

- **Ordered collection** accessed by **index**
- **Mutable**

Ordered, Indexed Collection

Ordered: The sequential order of the items inside will not change.

Indexed: The order begins with **0**.

0, 1, 2,
3, 4, 5

Mutable

Mutable: The items inside of a list can be changed.

Some things in programming will be **immutable** - they cannot be changed once defined.

0, 8, 2,
3, 4, 32



List Syntax

- Create lists using square brackets **[]**
- Separate items with **commas**

```
my_list = [1, 2, 3, 4, 5]
mixed_list = ["apple", 42,
True, 3.14]
empty_list = []
```

List Operations

Access and modify elements using **indexing**.

- Assigning an index location to a variable will make it the variable's value.

- `first_fruit = fruits[0]`

- An index location as a variable will reassign that location's value.

- `fruits[1] = "blueberry"`

```
fruits = ["apple", "banana", "cherry"]  
first_fruit = fruits[0] # "apple"  
fruits[1] = "blueberry" # ["apple",  
"blueberry", "cherry"]
```

List Operations

Basic list methods utilize arguments.

- `.append()` adds the passed value to the end of the list.
 - `fruits.append("orange")`
- `.remove()` removes the value at the passed index location.
 - `removed = fruits.pop(0)`

```
fruits.append("orange") # Adds to the
end
removed = fruits.remove("apple") #
Removes "apple"
```



List Operations

More specific list methods utilize **multiple arguments**. These are always separated by a comma.

- **.insert()** uses the passed location to know where to add the value.
- **.pop()** removes the value at the passed index location.

```
fruits.insert(1, "orange") # Adds to  
the "1" index location
```

```
removed = fruits.pop(0) # Removes  
"apple"
```

List Operations

You can use **colons** inside of **brackets** to extract a specific part of the list.

- **subset excludes 4**, and takes index 1 to 3.
- **last_three includes 3** and takes from index 3 until the end.
- **every_other** takes elements at **even indices** (0, 2, 4)

```
fruits = ["apple", "banana",  
"cherry", "pineapple", "mangosteen"]  
  
subset = fruits[1:4] # ["banana",  
"cherry", "pineapple"]  
  
last_three = fruits[-3:] #  
["cherry", "pineapple", "mangosteen"]  
  
every_other = fruits[::2] #  
["apple", "cherry", "mangosteen"]
```



List Best Practices

- **Homogenous:** Lists can have multiple data types, but best practice keeps them homogenous - meaning all as one data type.
 - `my_stuff = ["Notebook", 42, ["pen", "pencil"]]` would be a **heterogeneous** list, and not best practice.
- **Meaningful** slice variables: `first_three` says a lot more about the slice than `my_fruit`.



Topic

Dictionaries



Dictionaries

Dictionaries allow you to store multiple items as **key-value pairs**.

- **Unordered collection** accessed by **key**
- Keys are **immutable**, values are mutable.

Unordered Collection

Unordered: There is no sequential order of the items.

Instead of using indices to access values, you will use their **keys**.

```
"apple" : "green"
```

```
"banana" : "yellow"
```

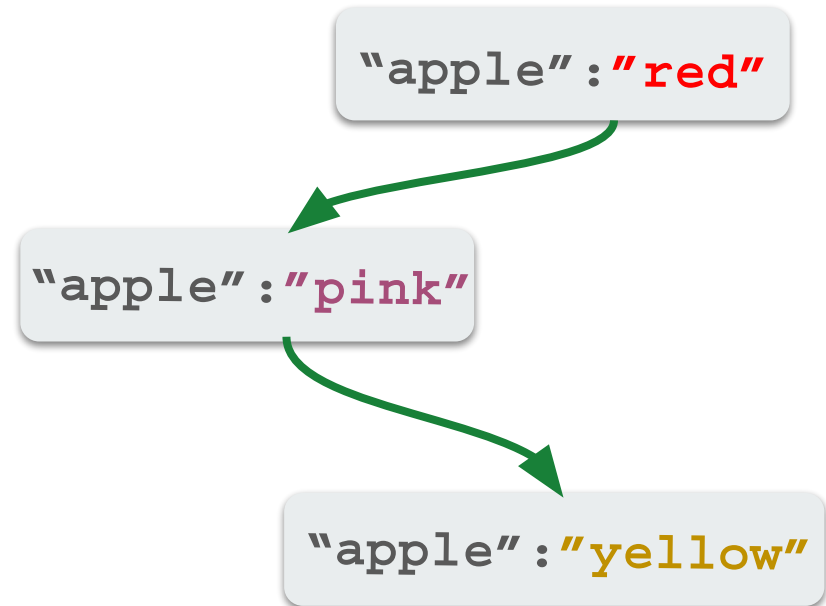
```
"cherry" : "red"
```

```
"tangerine" : "orange"
```

Immutable & Mutable

Immutable: Once defined, **keys cannot be changed**. This keeps the dictionary efficient and predictable.

However, their **values are mutable**, they can be changed.





Dictionary Syntax

- Create dictionaries using curly braces `{}`
- Format as **key: value** pairs
- Separate pairs with **commas**

```
person = {  
    "name": "Alex",  
    "age": 34,  
    "city": "Brookfield"  
}  
empty_dict = {}
```



Dictionary Operations

Most of your operations with dictionaries will use **keys**.

- **Bracket notation** can be used to add or change pairs.
- Combine bracket notation with **del** for removal.

```
student = {"name": "Emma",  
           "grade": 92,  
           "parking_permit": "Lot B"}
```

```
name = student["name"]  
# Access  
  
student["email"] = "emma@example.com"  
# Add  
  
student["grade"] = 95  
# Update  
  
del student["parking_permit"]  
# Remove
```



Dictionary Methods

Dictionaries have a number of methods commonly used.

For `keys()` and `values()`, it's important to note that these methods do not return lists.

```
student = {"name": "Emma",  
           "grade": 92,  
           "parking_permit": "Lot B"}
```

```
keys = student.keys()  
# Get all keys  
  
values = student.values()  
# Get all values  
  
has_permit = "parking_permit" in  
student  
# Checking if "parking_permit" is  
present
```



Nesting Dictionaries

Dictionaries can contain other dictionaries. This is useful for more complex information.

To access nested values, once again use **bracket notation**.

```
tenth_grade_lit = {  
    "student_1":  
        {"name": "Emma",  
         "grade": 92,  
         "parking_permit": "Lot B"},  
    "student_2":  
        {"name": "Gammell",  
         "grade": 56,  
         }  
}
```

```
# Accessing nested values  
gammell_grade =  
tenth_grade_lit["student_2"]["grade"]
```



Dictionary Best Practices

- **Heterogeneous:** Unlike lists, dictionaries can be mixed type, or heterogeneous.
 - However, best practice is to have the **keys** be the same data type.
- **Consistent** naming pattern: `student_1` and `student_2` are more predictable than `student_1` and `student_two`.



Topic

Loops



Loops

A **loop** is a control flow structure that allows code to be executed repeatedly based on a condition.

- **For** Loop
- **While** Loop
 - Control Statements

For Loops

Executes code for each item in a sequence.

- **for** item **in** collection:
 - do this ()

```
my_pets = ["Rex", "Necco", "Mars",  
"Perry Dog"]
```

```
for pet in my_pets:  
    print(f"I love {pet}")
```



```
I love Rex  
I love Necco  
I love Mars  
I love Perry Dog
```



While Loops

Executes code as long as condition is true.

- **while** condition:
 - do this ()

While it is dinner time, count every pet and call them.

Once every pet is counted and called, dinner is over.

```
my_pets = ["Rex", "Necco", "Mars",  
           "Perry Dog"]
```

```
dinner_time = true  
count = 0
```

```
while dinner_time:  
    print(f"{my_pets[count]} dinner!")  
    count += 1  
  
    if count >= len(my_pets):  
        dinner_time = false
```

Control Statements

You can include **control statements** to get more specific behavior.

- **Break** will end the iteration based on the given condition.
- **Continue** will skip the given condition in the iteration.

```
# Break
for pet in my_pets:
    if pet == "Mars":
        break
    print(f"Hello Mars!")

# Continue
for pet in my_pets:
    if pet == "Perry Dog":
        continue
    print(f"Meow, meow {pet}!")
```



Loop Best Practices

- Use the **best** loop for the task:
 - **For** loops when the iteration count is **known**.
 - **While** loops when the iteration count is **unknown**.
- **Simple** loop structures: While loops can be nested together, it increases the complexity and likelihood of error in your code.



Exercise

Python Lists And Dictionaries